

A Formalisation of Algorithms for Sorting Network

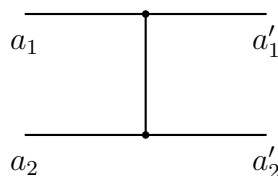
Laurent Théry
Laurent.Thery@sophia.inria.fr

Abstract

This notes explains how standard algorithms that construct sorting networks have been formalised and proved correct in the Coq proof assistant using the SSREFLECT extension.

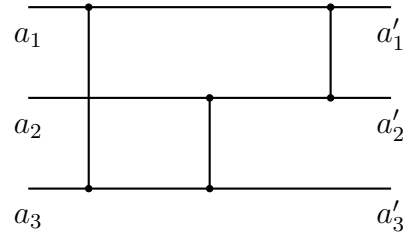
1 Introduction

A network is composed of a number of lines. By analogy to electronic circuit, each line has an input value before entering the network and an output value when leaving the network. The building block of a network is a comparator. A comparator connects two lines



A connector works as follows. The output value of the upper line is the minimum of the input values, $a'_1 = \min(a_1, a_2)$. The output value of the lower line is the maximum of the two lines, $a'_2 = \max(a_1, a_2)$.

A network is a collection of connectors. Here, we are interested into networks that sort their inputs, i.e. they return sorted outputs. An example of a network that sorts 3 inputs is the following



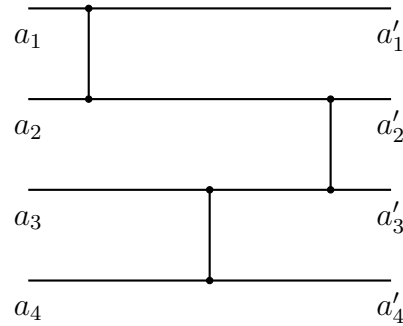
Whatever the initial values a_1 , a_2 and a_3 are, we have $a'_1 \leq a'_2 \leq a_3$. In the rest of the paper we are interesting in proving the correctness of some recursive algorithms that build sorting network. We first explain how we have formalised networks. Then, we present 3 algorithms:

- an algorithm that builds the bitonic sorting network;
- an algorithm that builds the odd-even merge sorting network;
- an algorithm that builds the odd-even exchange sorting network.

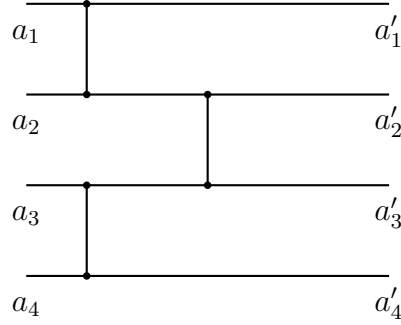
2 The formalisation

In the formalisation, we are using material that comes from the Mathematical Component Library. In order to make the presentation understandable by someone not familiar with this library, we summarize in the appendices [A](#), [B](#), [C](#) and [D](#) the notions that have been used for this formalisation.

To represent the state of lines, we are using the `tuple` type and are working on an arbitrary `orderedType A`. So if the network has m lines, the state of lines is represented by a m .-`tuple A`. We allow connectors to work simultaneously on several disjoint pairs of lines. If we consider the following sequence composed of 3 connectors



the first two are independent so can be performed in parallel while the third one must be kept separated as it shares some lines with the previous two. Making the parallelism explicit, we get the following drawing with only 2 connectors.



A connector is then encoded as a record that contains a function *clink* that takes a line (an element of I_m) and returns its associated line. The function is the identity for lines that are not connected. The requirement of the lines to be associated in disjoint pairs is encoded in the *cfinv* field which asks for *clink* to be involutive. A network is then a list of connectors.

```
Record connector (m : nat) := connector_of {
  clink : {ffun I_m => I_m};
  cfinv : [forall i, clink (clink i) == i]
}.
Definition network := seq (connector m).
```

An example of such a connector is the one that swaps the value of two line *i* and *j*. Its definition is done in three steps. We first define the link function, we prove that it is involutive, and we finally build the connector.

```
Definition clink_swap (i j : I_m) : {ffun I_m -> I_m} :=
  [ffun x => if x == i then j else if x == j then i else x].
Lemma clink_swap_proof (i j : I_m) :
  [forall k, clink_swap i j (clink_swap i j k) == k].
Definition cswap i j := connector_of (clink_swap_proof i j).
```

In the following, a variable c always represents a connector, n a network, s a sequence and t a tuple. The first operation on connector and network is the one that computes output values. The function $cfun$ applies a connector c to a tuple t and the function $nfun$ applies a network n to a tuple t .

```

Definition cfun  $c\ t :=$ 
  [tuple if  $i \leq \text{clink } c\ i$ 
    then min (tnth  $t\ i$ ) (tnth  $t\ (\text{clink } c\ i)$ )
    else max (tnth  $t\ i$ ) (tnth  $t\ (\text{clink } c\ i)$ ) |  $i < m$ ].

Definition nfun  $n\ t := \text{foldl } (\text{fun } t\ c \Rightarrow cfun\ c\ t)\ t\ n.$ 

```

The function $cfun$ performs the swap between values of connected lines, while $nfun$ simply iterates the application of $cfun$.

The first obvious property of connector and network is that they only permute their outputs. This is proved by the following theorems

```

Lemma perm_cfun  $c\ t : \text{perm\_eq } (cfun\ c\ t)\ t.$ 
Lemma perm_nfun  $n\ t : \text{perm\_eq } (nfun\ n\ t)\ t.$ 

```

Another interesting property is the regularity with respect to the order. If we take two arbitrary ordered types A and B and f a function from A to B that behaves well with the order ($f\ x \leq_B f\ y$ iff $x \leq_A y$) we have the following properties for min and max :

```

Lemma min_homo  $(x\ y : A) : f\ (\text{min } x\ y) = \text{min } (f\ x)\ (f\ y).$ 
Lemma max_homo  $(x\ y : A) : f\ (\text{max } x\ y) = \text{max } (f\ x)\ (f\ y).$ 

```

These properties can then be easily lifted at the level of connector and network.

```

Definition tmap  $f\ t := [\text{tuple } f\ (\text{tnth } t\ i) \mid i < m].$ 
Lemma tmap_connector  $c\ (t : m.\text{-tuple } A) : tmap\ f\ (cfun\ c\ t) = cfun\ c\ (tmap\ f\ t).$ 
Lemma tmap_network  $n\ (t : m.\text{-tuple } A) : tmap\ f\ (nfun\ n\ t) = nfun\ n\ (tmap\ f\ t).$ 

```

We are now ready to define the notion of sorting network. It is defined as a qualifier so we express the fact the n is a sorting network by the expression

"*n is sorting*". Thanks to the regularity with respect to the order, we can limit the definition of being a sorting network to the one of sorting all the boolean tuples. As, if we consider m lines, there are only a finite number of such tuples (2^m to be precise), this property is decidable and can be encoded as a boolean.

Definition *sorting* :=
 $[\text{qualify } n \mid [\text{forall } r : m.\text{-tuple bool, sorted}_{\leq} (nfun\ n\ r)]]$.

We now show that this boolean encoding captures exactly the usual notion of a sorting network: over any ordered type A , a network is sorting (in the boolean sense above) if and only if it sorts every tuple of elements of A . This is the zero-one principle.

One direction is easy. Provided A has at least two distinct elements, a network that sorts every A -tuple sorts in particular the boolean ones, so it is sorting in our sense.

Lemma *sorted_sorting* $n\ (x_1\ x_2 : A) :$
 $x_1 \neq x_2 \Rightarrow (\forall t : m.\text{-tuple } A, \text{sorted}_{\leq_A} (nfun\ n\ t)) \Rightarrow n \text{ is sorting}.$

Concretely, fix two distinct elements x_1, x_2 and let $f : \text{bool} \rightarrow A$ send **false** to $\min x_1\ x_2$ and **true** to $\max x_1\ x_2$. Given a boolean tuple r , the tuple $tmap\ f\ r$ has elements in A , so by hypothesis $nfun\ n\ (tmap\ f\ r)$ is sorted. Let $g : A \rightarrow \text{bool}$ send every element smaller than $\min x_1\ x_2$ to **false** and the rest to **true**. Then g is monotone and is a left inverse of f ($g\ (f\ b) = b$), so by *tmap_network*

$tmap\ g\ (nfun\ n\ (tmap\ f\ r)) = nfun\ n\ (tmap\ g\ (tmap\ f\ r)) = nfun\ n\ r$. Being the monotone image (under g) of a sorted tuple, $nfun\ n\ r$ is sorted; hence n is sorting. The converse is harder, and we establish it by contradiction.

Lemma *sorting_sorted* $n\ (t : m.\text{-tuple } A) : n \text{ is sorting} \Rightarrow \text{sorted}_{\leq_A} (nfun\ n\ t).$

Take an arbitrary A -tuple t of elements of A and let s be $(nfun\ n\ t)$. Suppose s is not sorted: there is an index i with $s[i] > s[i + 1]$. Let $h : A \rightarrow \text{bool}$

send every element strictly below $s[i]$ to **false** and the rest to **true**; h is monotone. Then $tmap\ h\ t$ is a boolean tuple, and by $tmap_network$

$$nfun\ n\ (tmap\ h\ t) = tmap\ h\ (nfun\ n\ t) = tmap\ h\ s$$

which by construction is not sorted : at positions $i, i + 1$ it holds **true** then **false**, since $h\ s[i] = \mathbf{true}$ and $h\ s[i + 1] = \mathbf{false}$. This contradicts n being sorting on boolean tuples, so s must be sorted.

Now, we are ready to build sorting networks. We first need building blocks. A key block is the one that glues together two networks: given a network n_1 with m_1 lines and a network n_2 with m_2 lines, it creates a network with $m_1 + m_2$ lines that behaves like n_1 on the top lines and n_2 on the bottom lines. There are different ways to do this. We favour the one that tries to fuze together connectors. This is the one that will be handy for building our sorting network later. So, at connector level, we have a connector c_1 with m_1 lines and a connector c_2 with m_2 lines and we want to build a connector of $m_1 + m_2$ lines. The first step is to build the associated *clink*. This requires some surgery with ordinals. Then, we need to prove that this new *clink* is involutive and we finally get our *cmerge* operation.

```

Definition clink_merge m1 m2 (c1 : connector m1) (c2 : connector m2) :=
  [ffun i => match split i with
    | inl x => lshift _ (clink c1 x)
    | inr x => rshift _ (clink c2 x)
  end].

Lemma clink_merge_proof m1 m2 (c1 : connector m1) (c2 : connector m2) :
  [forall i, (clink_merge c1 c2 (clink_merge c1 c2 i)) == i].

Definition cmerge m1 m2 (c1 : connector m1) (c2 : connector m2) :=
  connector_of (clink_merge_proof c1 c2).

```

Lifting this to network is easier. We create the sequence of pairs of connectors of n_1 and n_2 and on each of these pairs we apply *cmerge*.

```

Definition nmerge m1 m2 (n1 : network m1) (n2 : network m2) :=
  [seq cmerge i.1 i.2 | i <- zip n1 n2].

```

Note that this construction really makes sense if n_1 and n_2 have the same numbers of connectors. Otherwise the **zip** operation loses some connectors of the longest network. As a matter of fact, in the following, we mostly use the duplication operator that glues together two identical pieces.

Definition `ndup` m ($n : \text{network } m$) : $\text{network } (m + m) := \text{cmerge } n \ n$

Another way of gluing networks is the one based on parity. Given n_1 and n_2 , we build a network n whose even lines are ruled by n_1 and the odd ones by n_2 . We first need to introduce the division by 2 and the even and odd doubling at the level of ordinals.

Definition `idiv2` $m : 'I_m \Rightarrow 'I_m :=$
 if m is $m_1.+1$ then fun $i \Rightarrow \text{inZp } (i./2)$ else fun $i \Rightarrow i$.
 Definition `elift` $m : 'I_m \Rightarrow 'I_m :=$
 if m is $m_1.+1$ then fun $i \Rightarrow \text{inZp } (i.*2)$ else fun $i \Rightarrow i$.
 Definition `olift` $m : 'I_m \Rightarrow 'I_m :=$
 if m is $m_1.+1$ then fun $i \Rightarrow \text{inZp } (i.*2.+1)$ else fun $i \Rightarrow i$.

Then, we can introduce the parity merge for connectors.

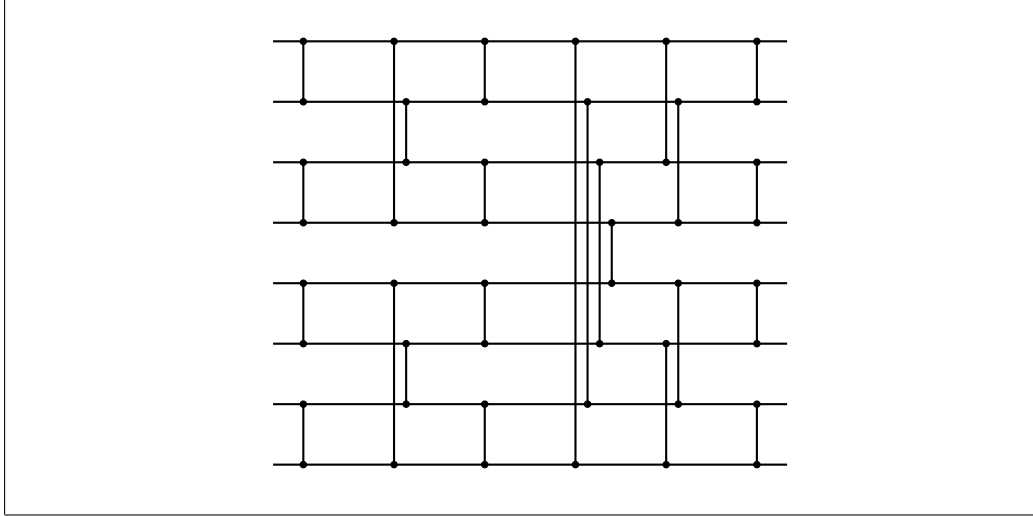
Definition `clink_eomerge` m ($c_1 : \text{connector } m$) ($c_2 : \text{connector } m$) :=
 [ffun $i : 'I_m \Rightarrow$
 if odd i then `olift` (`clink` c_2 (`idiv2` i))
 else `elift` (`clink` c_1 (`idiv2` i))].
 Lemma `clink_eomerge_proof` m ($c_1 : \text{connector } m$) ($c_2 : \text{connector } m$) :
 [forall i , (`clink_eomerge` c_1 c_2 (`clink_eomerge` c_1 c_2 i)) == i].
 Definition `ceomerge` m ($c_1 : \text{connector } m$) ($c_2 : \text{connector } m$) :=
`connector_of` (`clink_eomerge_proof` c_1 c_2).

Finally we can get the parity duplication

Definition `neomerge` m ($n_1 : \text{network } m$) ($n_2 : \text{network } m$) :=
 [seq `ceomerge` $i.1$ $i.2$ | $i <- \text{zip } n_1 \ n_2$].
 Definition `neodup` m ($n : \text{network } m$) : $\text{network } (m + m) := \text{neomerge } n \ n$.

3 Bitonic Sorter

Here is the version of the bitonic sorter for 8 lines.



It is composed of 6 connectors (the drawing of some links have been slightly shifted to the right so they don't overlap). The key ingredient of this network is the half-cleaner. It is a connector for $m + m$ lines, that links the line i to the line $i + m$ for $i < m$.

```

Definition clink_half_cleaner m : {ffun I_(m + m) ⇒ I_(m + m) } :=
  [ffun i =>
    match split i with
    | inl x => rshift _ x
    | inr x => lshift _ x
    end].
Lemma clink_half_cleaner_proof m :
  [forall i : I_(m + m), clink_half_cleaner _ (clink_half_cleaner _ i) == i].
Definition half_cleaner m := connector_of (clink_half_cleaner_proof m).

```

This connector has an interesting behaviour when given as input a so-called bitonic tuple. Technically, a sequence of elements is bitonic if there is one of its rotation that is increasing then decreasing.

```

Definition bitonic := [qualify s |
  [exists r : I_(size s).+1,
    exists n : I_(size s).+1,
    let s1 := rot r s in sorted ≤ (take n s1) && sorted ≥ (drop n s1)]].

```

Fortunately for sequences of booleans the characterisation is simpler : a sequence of booleans is bitonic if it has at most 2 flips.

```

Lemma bitonic_boolP (s : seq bool) :
  reflect (exists t,
    let: (b,i,j,k) := t in s = nseq i b ++ nseq j (~b) ++ nseq k b)
    (s is bitonic).

```

When applied to a bitonic sequence, the half-cleaner returns a tuple whose right half contains only **true** and the left half is bitonic or the left half contains only **false** and the right half is bitonic.

```

Lemma bitonic_half_cleaner m (t : (m + m).-tuple bool) :
  t is bitonic =>
  let t1 := cfun (half_cleaner m) t in
    ((take m t1 == nseq n false) && (drop m t1 is bitonic))
  ||
    ((drop m t1 == nseq n true) && (take m t1 is bitonic)).

```

The proof proceeds by case analysis. As the tuple contains only 2 flips, there are two easy cases when these two flips are both in a single half. When it is in the left half, we have

left half	b	b	b	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	b	b	b
right half	b	b	b	b	b	b	b	b	b	b
min	b	b	b	F	F	F	F	b	b	b
max	b	b	b	T	T	T	T	b	b	b

so the property holds. By symmetry this is the same if the two flips are on right half.

left half	b	b	b	b	b	b	b	b	b	b
right half	b	b	b	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	b	b	b
min	b	b	b	F	F	F	F	b	b	b
max	b	b	b	T	T	T	T	b	b	b

In the remaining cases, each half has a flip. Suppose the flip in the left half occurs first, we have:

left half	b	b	b	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$
right half	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	b	b	b
min	F	F	F	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	F	F	F
max	T	T	T	$\bar{b}$	$\bar{b}$	$\bar{b}$	$\bar{b}$	T	T	T

and the property holds again. Finally the flip in the right half occurs first, we have

left half	b	b	b	b	b	b	b	$\bar{b}$	$\bar{b}$	$\bar{b}$
right half	$\bar{b}$	$\bar{b}$	$\bar{b}$	b	b	b	b	b	b	b
min	F	F	F	b	b	b	b	F	F	F
max	T	T	T	b	b	b	b	T	T	T

This ends the proof.

The next observation is that if we recursively apply on the resulting halves the half-cleaner, we end up getting a sorted list : we progressively add **false** on the left part or **true** on the right one. Being able to perform this recursion on halves implies that the initial number of lines must be a power of 2. In our case, in order to insert a half-cleaner we need to have a type of the form *connector* ($m + m$). This means that it is mandatory for the typechecker to succeed that 2^{m+1} converts to $2^m + 2^m$. This is not the case with the exponential function of the library. So we define our own version that we write ‘ 2^n ’ in the following.

```
Fixpoint ‘2m := if m is m1.+1 then ‘2m1 + ‘2m1 else 1.
```

We can then define the recursive function.

```
Fixpoint half_cleaner_rec m : network ‘2m :=
  if m is m1.+1 then half_cleaner ‘2m1 :: ndup (half_cleaner_rec m1)
  else [::].
```

We can then easily prove its expected behaviour.

```

Lemma sorted_half_cleaner_rec m (t : '2m .-tuple bool) :
  t is bitonic  $\Rightarrow$  sorted  $\leq$  (nfun (half_cleaner_rec m) t).

```

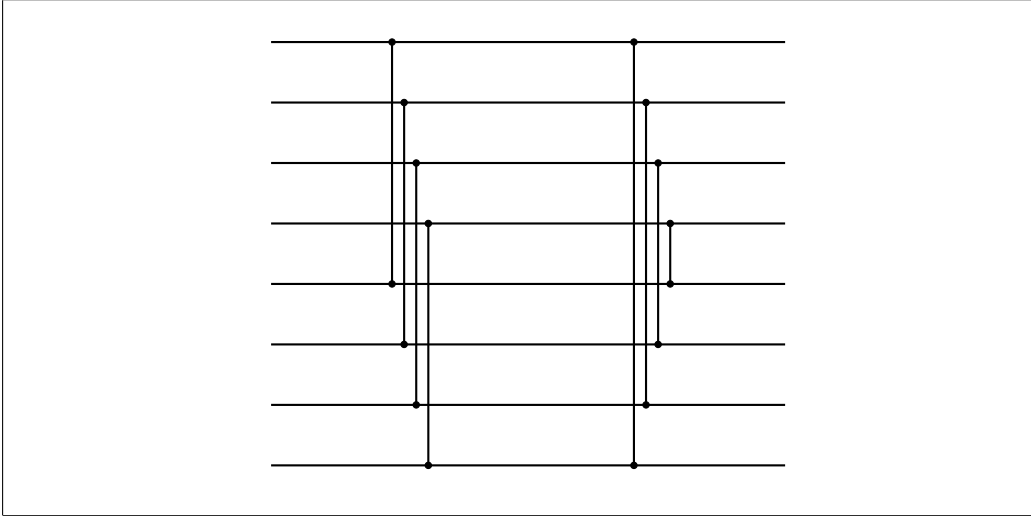
and show that it is logarithmic and creates a network of m connectors.

```

Lemma size_half_cleaner_rec m : size (half_cleaner_rec m) = m.

```

The recursive half-cleaner requires to have a bitonic entry. If we try to build a recursive algorithm, calling it first on the top-half lines and then on the bottom-half lines, we get two sorting outputs. Gluing them directly does not give a bitonic entry. There are possibly too many flips. Each half that is sorted contains potentially a flip and there is the potential flip at their intersection. Instead, the trick is to glue them together but reversing the second one. This leads to a bitonic entry. So, a reverse version of the half-cleaner is created that performs this reversal. Graphically, it looks like this.



On the left-hand side there is the standard half-cleaner. On the right-hand side there is the reverse version where the link to the bottom lines have been reversed. For example, the line 1 is linked to the line 5 on the left part. It is now linked to the line 8 on the right part. There is a **rev_ord** function for ordinals. We use it to implement the reverse half-cleaner, so the line i is connected to line $(m + 1) - i$:

```

Definition clink_rhalf_cleaner m : {ffun I_m  $\Rightarrow$  I_m} := [ffun i  $\Rightarrow$  rev_ord i].
Lemma clink_rhalf_cleaner_proof m :
  [forall i : I_(m + m), clink_rhalf_cleaner _ (clink_rhalf_cleaner _ i) == i].
Definition rhalf_cleaner m := connector_of (clink_rhalf_cleaner_proof m).

```

Now, we can use the reverse half-cleaner before calling the recursive half-cleaner.

```

Definition rhalf_cleaner_rec n : network '2^n :=
  if n is n1.+1 then rhalf_cleaner '2n1 :: ndup (half_cleaner_rec n1)
  else [::].

```

The call to the reverse half-cleaner produces on the top-half lines either only **true** values so there is no problem or a reverse of a bitonic but it is also ok, the reverse of a bitonic is a bitonic. The same holds for the bottom-half lines. So, we get the expected theorem.

```

Lemma sorted_rhalf_cleaner_rec m (t : '2m.+1.-tuple bool) :
  sorted  $\leq$  (take '2m t)  $\Rightarrow$  sorted  $\leq$  (drop '2m t) - >
  sorted  $\leq$  (nfun (rhalf_cleaner_rec m.+1) t).

```

Now, we can build the recursion

```

Fixpoint bsort m : network '2m :=
  if m is m1.+1 then ndup (bsort m1) ++ rhalf_cleaner_rec m1.+1
  else [::].

```

and get the final results.

```

Lemma sorting_bsort m : bsort m is sorting.
Lemma size_bsort m : size (bsort m) = (m * m.+1)./2.

```

Here is the complete code of the algorithm.

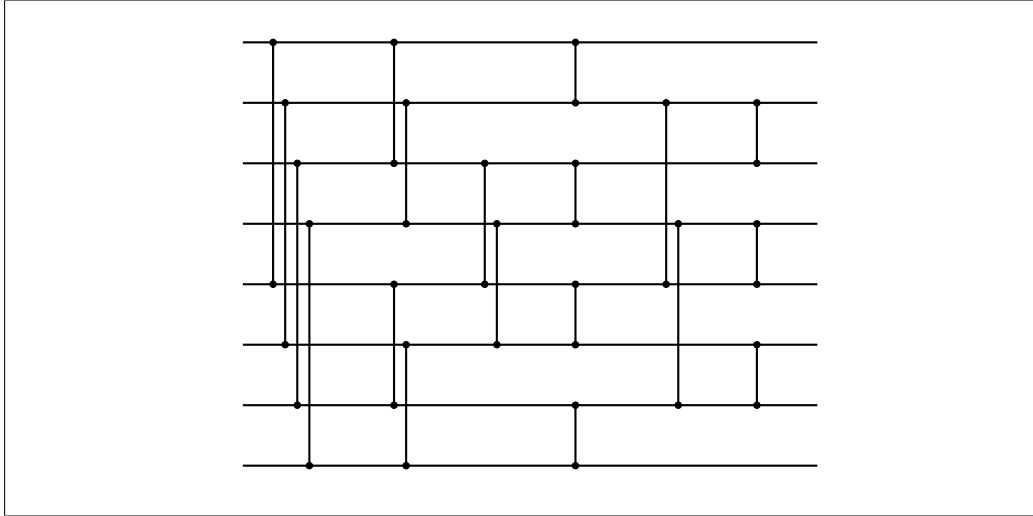
```

Fixpoint half_cleaner_rec m : network '2m :=
  if m is m1.+1 then half_cleaner '2m1 :: ndup (half_cleaner_rec m1)
  else [::].
Definition rhalf_cleaner_rec n : network '2n :=
  if n is n1.+1 then rhalf_cleaner '2n1 :: ndup (half_cleaner_rec n1)
  else [::].
Fixpoint bsort m : network '2m :=
  if m is m1.+1 then ndup (bsort m1) ++ rhalf_cleaner_rec m1.+1
  else [::].

```

4 Knuth's Exchange Odd Even Sorter

Here is the drawing of the odd-even sorter.



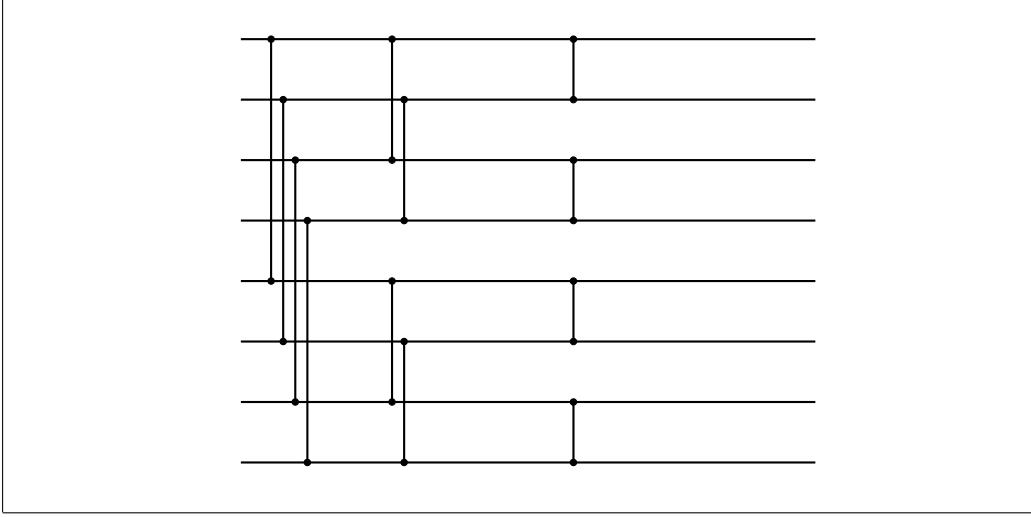
This is still a recursive algorithm but this time it is not based on a top-half, bottom-half partition but an even and odd partition. We add them as basic operations on sequences.

```

Fixpoint etake s :=
  if s is a :: s1 then a :: (if s1 is _ :: s2 then etake s2 else [::])
  else [::].
Definition otake s := if s is _ :: s1 then etake s1 else [::].

```

There are two components of this sorter. The first one is the one that connects each even line to one of its odd neighbours.



We first have 4 copies with jump 4 then 2 copies with jump 2 finally 1 copy with jump 1. The copy with jump 1 on the right shows the structure: even lines are linked to their down neighbour. In order to encode it, we need to introduce the notion of neighbour for ordinals.

```

Definition inext m : I_m -> I_m :=
  if m is m1.+1 then fun i => inZp (if i == m1 then i else i.+1)
  else fun i => i.
Definition ipred m : I_m -> I_m :=
  if m is m1.+1 then fun i => inZp (i.-1) else fun i => i.

```

We can define the connector.

```

Definition clink_eswap m: {ffun I_m -> I_m} :=
  [ffun i : I_ - => if odd i then ipred i else inext i].
Lemma clink_eswap_proof m :
  [forall i : I_m, clink_eswap _ (clink_eswap _ i) == i ].
Definition ceswap m := connector_of (clink_eswap_proof m).

```

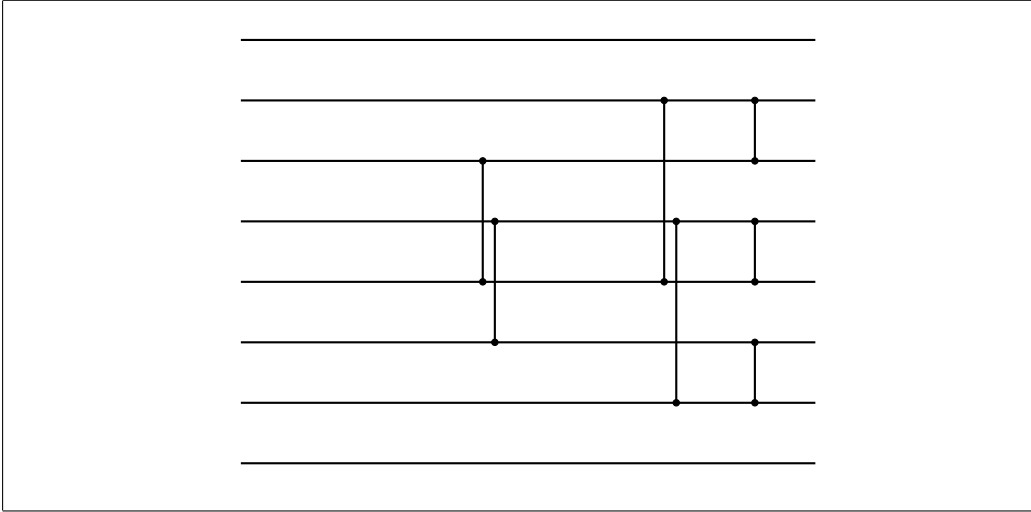
If we look at the effect of applying this connector to a tuple of booleans, if the even lines and the odd lines are sorted, this property is preserved plus the even part contains more **false** than the odd part:

```

Definition noF (s : seq bool) := count (fun b => ~~b) s.
Lemma sorted_eswap m (t : (m + m).-tuple bool) :
  sorted≤ (etake t) -> sorted≤ (otake t) ->
  let t1 := cfun ceswap t in
  [^ sorted≤ (etake t1),
   sorted≤ (otake t1) &
   noF (otake t1) ≤ noF (etake t1)].

```

The second connector is the one that connects the odd lines with a k jump (k is odd) to the even lines.



There are 2 copies with jump 1, then one copy with jump 3 and one copy with jump 1. Again, we define first the operation on ordinals.

```

Definition iadd m k : I_m -> I_m :=
  if m is m1.+1 then fun i => inZp (if k + i ≤ m1 then k + i else i)
  else fun i => i.
Definition isub m k : I_m -> I_m :=
  if m is m1.+1 then fun i => inZp (if k ≤ i then i - k else i)
  else fun i => i.

```

We then create the connector.

```

Definition clink_odd_jump m k : {ffun I_m -> I_m} :=
  if odd k then [ffun i => if odd i then iadd k i else isub k i ]
  else [ffun i => i].
Lemma clink_odd_jump_proof m k :
  [forall i : I_m, clink_odd_jump _ k (clink_odd_jump _ k i) == i].
Definition codd_jump m k := connector_of (clink_odd_jump_proof m k).

```

This time, the **false** values are moving from the even lines to the odd lines and we can quantify exactly how much.

```

Lemma sorted_odd_jump m (t : (m + m).-tuple bool) i k :
  odd k -> i <= (uphalf k).*2 ->
  sorted≤ (etake t) -> sorted≤ (otake t) ->
  noF (etake t) = noF (otake t) + i ->
  let j := i - uphalf k in
  let t1 := cfun (codd_jump k) t in
  [∧ sorted≤ (etake t1),
   sorted≤ (otake t1) &
   noF (etake t1) = noF (otake t1) + (i - j.*2)].

```

Note that here we make use of the fact that $m - n = 0$ if $n \geq m$.

Now, the idea of the algorithm is to reduce the difference between the number of **false** between the odd and the even part so that the list becomes sorted.

```

Lemma sorted_etake_otake m (t : (m + m).-tuple bool) :
  sorted≤ (etake t) -> sorted≤ (otake t) ->
  noF (otake t) ≤ noF (etake t) ≤ (noF (otake t)).+1 ->
  sorted≤ t.

```

This is done by recursively halving the jump and we get the expected result.

```

Fixpoint knuth_jump_rec m k r : network m :=
  if k is k1.+1 then codd_jump r :: knuth_jump_rec m k1 (uphalf r).-1
  else [::].
Lemma sorted_knuth_jump_rec m (t : (m + m).-tuple bool) k :
  sorted≤ (etake t) -> sorted≤ (otake t) ->
  noF (otake t) ≤ noF (etake t) ≤ noF (otake t) + '2k ->
  sorted≤ (nfun (knuth_jump_rec (m + m) k ('2k).-1) t).

```

We can now put together the recursion, the even swap and the recursive jump to get the sorter.

```

Fixpoint knuth_exchange m : network '2m :=
  if m is m1.+1 then
    neodup (knuth_exchange m1) ++ ceswap :: knuth_jump_rec '2m m1 (('2m1).-1)
  else [::].
Lemma sorting_knuth_exchange m : knuth_exchange m is sorting.
Lemma size_knuth_exchange m : size (knuth_exchange m) = (m * m.+1)/2.

```

Here is the complete code of the algorithm.

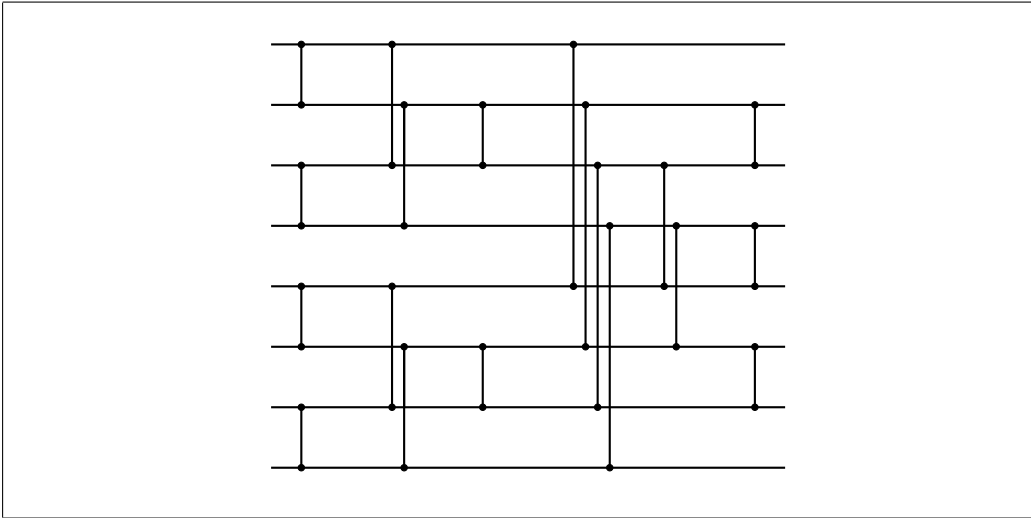
```

Fixpoint knuth_jump_rec m k r : network m :=
  if k is k1.+1 then codd_jump r :: knuth_jump_rec m k1 (uphalf r).-1
  else [::].
Fixpoint knuth_exchange m : network '2m :=
  if m is m1.+1 then
    neodup (knuth_exchange m1) ++ ceswap :: knuth_jump_rec '2m m1 (('2m1).-1)
  else [::].

```

5 Batcher's Odd Even Sorter

The last algorithm we are going to consider is using both the top-bottom recursion and an even-odd recursion. For 8 lines, we get.



This sorter uses only two connectors. The *cswap* connector is used in the base case for sorting two lines. The *codd_jump* connector with a jump of one is used at the end of the iteration to get the sorted result when it is sure that the numbers of **false** of the even part exceeds of at most 2 the ones of the odd part.

```

Definition batcher_merge m : connector m := codd_jump 1.
Lemma sorted_batcher_merge m (t : (m + m).-tuple bool) :
  noF (otake t) ≤ noF (etake t) ≤ (noF (otake t)).+2 ->
  sorted≤ (etake t) -> sorted≤ (otake t) ->
  sorted≤ (cfun batcher_merge t).

```

In order to sort the odd and even parts, the sorter uses an odd and even recursion.

```

Fixpoint batcher_merge_rec_aux m : network '2m.+1 :=
  if m is m1.+1 then rcons (neodup (batcher_merge_rec_aux m1)) batcher_merge
  else [:: cswap ord0 ord_max].
Definition batcher_merge_rec m :=
  if m is m1.+1 then batcher_merge_rec_aux m1 else [::].

```

The idea is the following. If the top-half and the bottom-half are sorted, their respective odd and even part differ by at most one in the number of **false** (the odd part being the smallest). When taking the odd part and even part of all the lines, it then differs by at most two. After sorting them, we are within the conditions of theorem *sorted_batcher_merge*. As having top-half and bottom-half sorted is preserved by taking the odd or the even part, we get the following theorem.

```

Lemma sorted_nfun_batcher_merge_rec m (t : '2m.+1.-tuple bool) :
  sorted≤ (take '2m t) -> sorted≤ (drop '2m t) ->
  sorted≤ (nfun (batcher_merge_rec_aux m) t).

```

We are almost done. We can use top-bottom recursion to fulfill the conditions of theorem *sorted_nfun_batcher_merge_rec*.

```

Fixpoint batcher m : network '2m :=
  if m is m1.+1 then ndup (batcher m1) ++ batcher_merge_rec m1.+1
  else [::].

```

and we get the expected properties.

```

Lemma sorting_batcher m : batcher m is sorting.
Lemma size_batcher m : size (batcher m) = (m * m.+1)./2.

```

Here is the complete code of the algorithm.

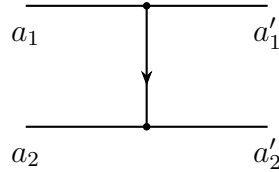
```

Fixpoint batcher_merge_rec_aux m : network '2m.+1 :=
  if m is m1.+1 then rcons (neodup (batcher_merge_rec_aux m1)) batcher_merge
  else [:: cswap ord0 ord_max].
Definition batcher_merge_rec m :=
  if m is m1.+1 then batcher_merge_rec_aux m1 else [::].
Fixpoint batcher m : network '2m :=
  if m is m1.+1 then ndup (batcher m1) ++ batcher_merge_rec m1.+1

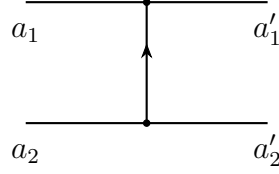
```

6 Extension

A standard extension is to use oriented comparator. Graphically, the orientation indicates which line gets the maximum of the two lines. This means that, so far, we have been using comparator with the arrow down.



Instead, with the arrow up, the value of the upper line is the maximum of the input values, $a'_1 = \max(a_1, a_2)$, and the output value of the lower line is the minimum of the two lines, $a'_2 = \min(a_1, a_2)$.



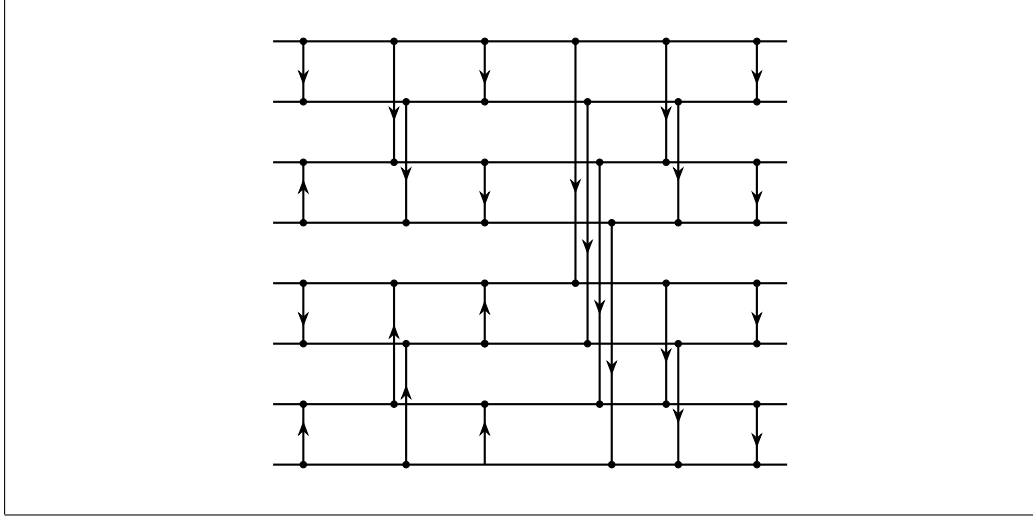
In our formalisation, this means that we need to add an extra component that keeps the orientation of the link. This is the field `cflip` that associates a boolean to every line. The field `cflipinv` ensures that associated lines have identical flip value.

```
Record connector (m : nat) := connector_of {
  clink : {ffun I_m -> I_m};
  cflip : {ffun I_m -> bool};
  cfinv : [forall i, clink (clink i) == i];
  cflipinv : [forall i, cflip (clink i) == cflip i]}.
```

These modifications change the way we define *cfun*

```
Definition cfun c t :=
  [tuple let min := min (tnth t i) (tnth t (clink c i)) in
    let max := max (tnth t i) (tnth t (clink c i)) in
    if i ≤ clink c i then if cflip c i then max else min
    else if cflip c i then min else max | i < m].
```

The main algorithm that benefits from having this new capability is the bitonic sorter.



The drawing is more regular since it uses the *half_cleaner* connector only.

```

Lemma cflip_default m (clink : {ffun I_m -> I_m}) (b : bool) :
  [forall i, [ffun => b] (clink i) == [ffun => b] i] .
Definition half_cleaner b m :=
  connector_of (clink_half_cleaner_proof m) (cflip_default (clink_half_cleaner m) b).

```

It is now possible to write a version of the bitonic sorter *bfsort* that uses the flip.

```

Fixpoint half_cleaner_rec b m : network '2^m :=
  if m is m_1.+1 then half_cleaner b '2^m_1 :: ndup (half_cleaner_rec b m_1)
  else [::].

Fixpoint bfsort (b : bool) m : network '2^m :=
  if m is m_1.+1 then nmerge (bfsort b m_1) (bfsort (~b) m_1) ++
    half_cleaner_rec b m_1.+1
  else [::].

Lemma size_bfsort b m : size (bfsort b m) = (m * m.+1) ./ 2.
Lemma sorting_bfsort m : bfsort false m is sorting.

```

7 Conclusion

In this paper, we have shown how to formalise different sorting algorithms for networks. We have been following mostly what is presented in chapter 27

of [2]. Another source of inspiration was [1]. We have been using intensively the zero-one principle. Most proofs are done manipulating booleans. It looks a bit like magic. The formalisation is available at

<https://github.com/theyry/mathcomp-extra>

It consists of 5 files. The file `more_tuple` contains some addition to the Mathematical Library. It is 1000-line long. The file `nsort` contains the definition of network and some basic connectors. It is 700-line long. The file `bitonic` deals with the bitonic sorter. It is 500-line long. The file `bjsort` deals with the exchange sorter. It is 200-line long. The file `batcher` deals with the exchange sorter. It is 200-line long.

From the specification point of view, we believe that having explicit networks and using dependent types for this gives us a very concise presentation of the algorithms. All the usual index manipulations are hidden inside the *ndup* and *neodup* building blocks. From the proving point of view, the difficult part in the bitonic sort is proving the specification of the half-cleaner. For the other sorters, the only delicate thing is the manipulation of *codd_jump* connectors. The introduction of the function *noF* makes the specification and proof easier.

References

- [1] Ana Bove and Thierry Coquand. Formalising bitonic sort in type theory. In *TYPES*, volume 3839, pages 82–97. Springer, 2004.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, 2nd Edition*. MIT Press, 2001.

A Basic

<code>x = y</code>	propositional equality between x and y
<code>x == y</code>	boolean equality between x and y that must belong to an <code>eqType</code>
<code>reflect P b</code>	equivalence between the propositions P and $(b = \text{true})$
<code>n.+1</code>	add one to the natural number n
<code>n.*2</code>	double the natural number n
<code>n./2</code>	half the natural number n
<code>uphalf n</code>	half the natural number $n + 1$
<code>odd n</code>	<code>true</code> if n is odd, <code>false</code> otherwise
<code>(l, r)</code>	the pair composed of r and l
<code>p.1</code>	the first component of the pair p
<code>p.2</code>	the second component of the pair p
<code>[qualify x P]</code>	if $A := [\text{qualify } x \mid P]$, x is A is equivalent to P

B Fintype

<code>[forall x, P]</code>	P (in which x can appear) is true for all values of x x must range over a <code>finType</code>
<code>I_n</code>	the finite subType of integers $i < n$
<code>ord0</code>	the $i : I_n.+1$ with value 0
<code>ord_max</code>	the $i : I_n.+1$ with value n
<code>inZp</code>	the natural projection from <code>nat</code> into the integers mod p , represented as <code>'I_p</code> . Here p is implicit, but must be of the form $n.+1$
<code>rev_ord i</code>	the complement to $n.-1$ of $i : I_n$
<code>lshift n j</code>	the $i : 'I_n$ with value $j : 'I_m$
<code>rshift m k</code>	the $i : 'I_n$ with value $m + k$, $k : 'I_n$
<code>split i</code>	i has type <code>'I_n</code> it returns <code>inl j</code> when there exists j such that $i = \text{lshift } n \ j$ it returns <code>inr k</code> when there exists k such that $i = \text{rshift } m \ k$
<code>{ffun A ⇒ B}</code>	type for functions with a finite domain (A should be a <code>finType</code>)
<code>[ffun x => E]</code>	definition of a function with a finite domain (x may appear in E)

C Sequences

<code>[::]</code>	the empty sequence
<code>x :: s</code>	the sequence starting with x followed by s
<code>rcons s x</code>	the sequence starting with s and ended by x
<code>[seq E i <- l]</code>	the sequence with general term E (i in l and bound in E)
<code>size s</code>	the number of items (length) in s
<code>count P s</code>	the number of items of s for which P holds
<code>nseq n x</code>	a sequence of n x 's
<code>head x₀ s</code>	the head (zero'th item) of s if s is non-empty, else x_0
<code>behead s</code>	s minus its head
<code>last x s</code>	the last element of $x :: s$ (which is non-empty)
<code>belast x s</code>	$x :: s$ minus its last item
<code>s₁ ++ s₂</code>	the concatenation of s_1 and s_2
<code>take n s</code>	the sequence containing only the first n items of s (or all of s if $\text{size } s \leq n$)
<code>drop n s</code>	s minus its first n items (<code>[::]</code> if $\text{size } s \leq n$)
<code>rot n s</code>	s rotated left n times (or s if $\text{size } s \leq n$)
<code>zip s t</code>	itemwise pairing of s and t (dropping any extra items) <code>[::(x₁, y₁); ...; (x_{mn}, y_{mn})]</code> with $mn = \min n m$
<code>foldl f a s</code>	the left fold of s by f , i.e. $f (f \dots (f a x_1) \dots x_{n-1}) x_n$
<code>perm_eq s₁ s₂</code>	s_2 is a permutation of s_1 , i.e., s_1 and s_2 have the items (with the same repetitions), but possibly in a different order
<code>sorted_e s</code>	s is an e -sorted sequence: either $s = [::]$, $s = [::x]$, or $s = x :: y :: s_1$ with $e x y$ and $(y :: s_1)$ is e -sorted

D Tuple

<code>n.-tuple T</code>	the type of n-tuples of elements of type T
<code>[tuple E i < n]</code>	the <code>n.-tuple</code> with general term E ($i : \mathbf{I_n}$ is bound in E)
<code>tnth t i</code>	the i 'th component of t , where $i : \mathbf{I_n}$